



Sorting Fruits and Vegetables with Neural Networks (CNN, DNN, MPL)

Justin Ogle
San Jose State University
College of Engineering
EE258 - Sensor Technology and Principles
Summer 2023
Charles Choo, Ph.D
9/2/2023

Abstract

The purpose of this project is to identify and sort fruits and vegetables by use of neural networks. The main code base and dataset I used comes from the Kaggle web site under the “Fruits and Vegetables Image Recognition Dataset.” [1] I modified the program to sort the images after initial the identification step to sort and classify if these are a fruit, vegetable, or not a type of produce. Also translated the code from a Convolutional Neural Network (CNN) to a Deep Neural Network (DNN) and (MLP) Multilayer Perceptron to learn if these other neural networks can perform better than the other. Plus, this has helped me gain a better understanding of the python coding language.

Table of Contents

Abstract	1
Table of Contents	2
Problem and Dataset Description	3
Preprocessing	4
Baseline Model	5
Models Utilized	15
Model Improvements	13
Discussions & Conclusions	20
References	22

Problem and Dataset Description

The problem in life, fruits and vegetables always arrive to us via the same distribution methods and pathways. To save time on getting expediently to the table, I am implementing a neural network to classify a multiclass image dataset problem of sorting individual types of fruits and vegetables.

The other reason I am interested in pursuing this topic is one time I was at the grocery store and my girlfriend was tired and went to the car to take a nap while I was in the checkout line. When it was my turn to checkout, one item of produce did not have the sticker on itself. I thought it was a beet, but the employee did not know either. I did not know how to use a smartphone to identify the produce and the employees are not allowed to use their phones while they are working. An hour later my girlfriend wakes up and finds me in the front of a long angry line just to point out that this item of produce was indeed a rutabaga. I want to see how a program can identify an image to see if it is produce or not, then maybe implement these on check out scanners to see and weigh what it is and price accordingly if the sticker is not on.

The initial dataset is from the Kaggle web site under the “Fruits and Vegetables Image Recognition Dataset” which has thirty-six classes of produce, eleven Fruits, and twenty Vegetables. Their intended use for the code is to identify fruits or vegetables for recipes. Each folder should contain these images: ten in test, Near 100 in train, ten in validation.

The 11 fruits in the initial dataset are: banana, apple, pear, grapes, orange, kiwi, watermelon, pomegranate, pineapple, mango, and lemon.

The 25 vegetables in the initial dataset are: cucumber, carrot, capsicum, onion, potato, tomato, radish, beetroot, cabbage, lettuce, spinach, soy bean, cauliflower, bell pepper, chili pepper, turnip, corn, sweetcorn, sweet potato, paprika, jalapeño, ginger, garlic, peas, and eggplant.

After using a visualization tool, I have found out that:

- The Test data is at a 97.22% each Class has ten images with the outlier of a banana only has nine images.
- The Train data has only two Classes with 100 images (grapes & peas) while all the rest are in range of sixty eight to 100 images.

- o The outlier for train data is it contains more and random amounts of images and is supposed to run against the test and validation images.
- The Validation data is at a 75% each Class has ten images.
 - o The outlier is nine images containing the amount of nine (banana, beetroot, carrot, cauliflower, grapes, jalapeño, lettuce, orange, and radish).
- Overall, there must have been some disagreements with test and validation upon dataset set-up.

Pre-Processing

The way I pre-processed the dataset is by adding more produce to their respective directories and an unknown directory in order to increase the program accuracy. The idea is to expand the directories to identify more produce. Also, to see what the program can do if some fruit have similar attributes to others. This makes the fifty-one classes to work with. This makes a total of twenty-one fruits, twenty-nine vegetables, and one unknown class set to catch what is not produce.

The ten additional fruits in the dataset are: blackberry, blueberry, cherry, dragon fruit, honey dew, jackfruit, papaya, plum, raspberry, and strawberry.

The four additional vegetables in the dataset are: mushroom, olive, parsnip, and rutabaga.

These additional directory images were selected randomly from a google image search.

Four directories are made: test, training, validation, and imgTest.

imgTest is the image test directory. This is where the individual will place the images that they want to check to see if it is produce or not in alphabetical order. For both the baseline and final test there will be one of each item supposedly from each classification but randomly selected if the image is real, toy/figurine/costume, or drawing of produce. The reason for this is to observe the accuracy of all classifications.

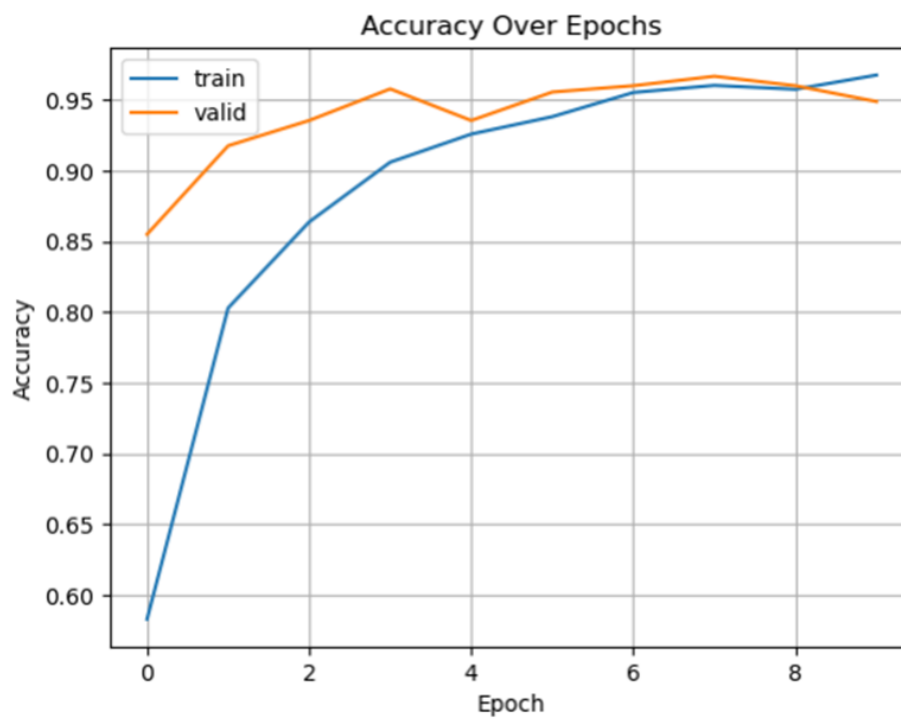
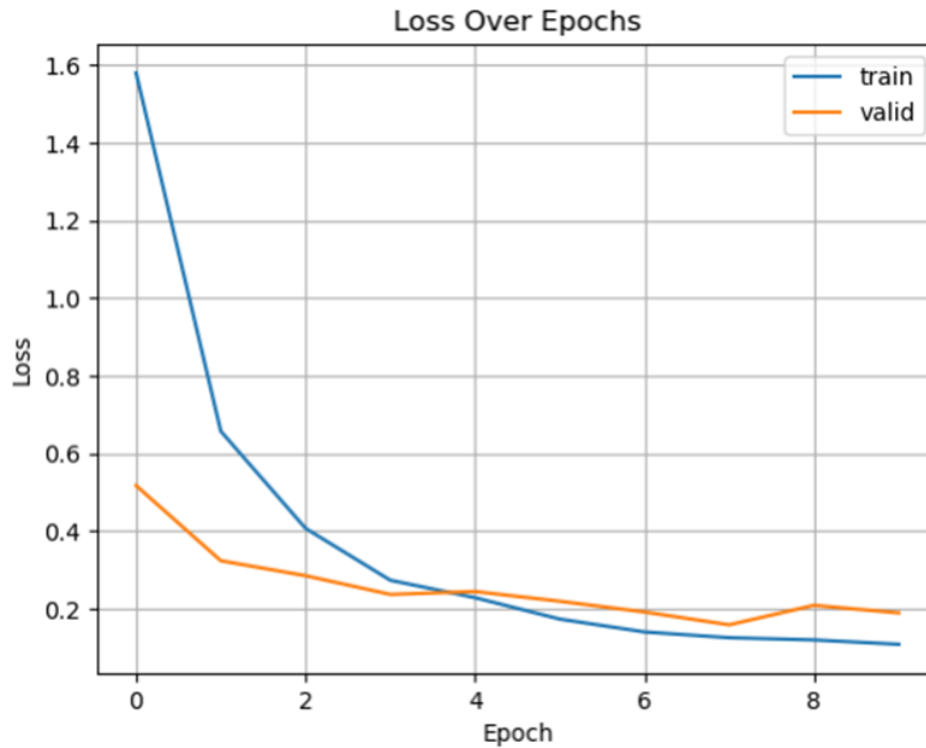
I took the base program from “Fruits and Vegetables Image Recognition Dataset” and found any line of code that was contradictory or not useful. I did find a few of these. I got rid of the “(os.path.join(dirname, filename))” due to not wanting to see a long list of the images in each directory and waiting on the output time. Change the num_class size from 36 to 51. While troubleshooting the code, I would reduce the epochs size to a lower amount to reduce producing time. Added a grid and title to the “Loss Over Epochs” and “Accuracy Over Epochs” graphs to make reading the results and identifying.

I am using the CNN, DNN, and MLP neural networks. CNN is preferred for image processing. DNN is a complex deep learning system that might handle this task. MLP is a three-layer system with one of the layers hidden.

Baseline Model

Each of the baseline models operated with epochs at 10 and the batch size at 32.

My baseline model is a Convolutional Neural Network (CNN) which is a preferred model for image processing. The image data is augmented by use of an image data generator which rescales at a 1/255 scale, shears at a 0.2 range, zooms in 0.2, and then horizontally flips. The epochs are set at 10. Each test generator is set to their own directory, the target size is set to 224 x 224, the batch size at 32, the class mode is categorical, and this does not shuffle. The tensor flow keras layers used are GlobalAveragePooling2D and Dense. The tensor flow keras model used is Model. The GlobalAveragePooling2D layer is to reduce spatial dimensions and flatten the output. The Dense layer has 1024 units and a ReLU activation function. The final Dense layer with num_classes units and softmax activation for classification. The model is set by specifying inputs and outputs. Then the program compiles the model with an Adam optimizer, categorical cross-entropy loss, and accuracy metric. After this the model is trained by using the data history and begins to calculate the set number of epochs. A data frame is created to store the data history of the epoch calculations. The plots are soon created. Both loss over epochs and accuracy over epochs.



The predictions are generated from a model predict test generator. The data is put into an array by using the np.argmax function. Now it will print the class indices.

Essentially at this point, the program investigates the assigned directory for images. It takes the images to refigure the dimensions and even flip it. Converts the image into a number after performing convolutions to match what number it relates to. Then it outputs a list and descriptor of the trained data in order that it recognizes the data and directories.

The variable `pred_classes` are made to define the mapping of class indices to class names. The map predicted indices to class names are set. Using the sklearn metric, the classification report is generated.

The results of the classification report are as follows:

	Precision	Recall	F1-Score	Support
Accuracy			0.95	474
Macro Average	0.95	0.96	0.95	474
Weighted Average	0.96	0.95	0.95	474

This last bit of code is where the test image is processed. The directory is defined. The augmentation is set with the batch set as one since each test is an individual picture. New predictions will be made from the additional tests. The predictions are mapped. Then the image is classified as either a fruit, vegetable, or unknown. The unknown category will print out as “Not a type of produce: [‘unknown’, ... ect]”.

The output will be a list of what is identified then how they are sorted.

CNN Baseline Results

What are these images? ['watermelon', 'cherry', 'unknown', 'apple', 'orange', 'beetroot', 'raspberry', 'sweetpotato', 'cauliflower', 'cabbage', 'soy beans', 'corn', 'jackfruit', 'chilli pepper', 'papaya', 'bell pepper', 'cucumber', 'carrot', 'cherry', 'ginger', 'jalepeno', 'peas', 'grapes', 'garlic', 'orange', 'kiwi', 'raddish', 'plum', 'potato', 'eggplant', 'strawberry', 'paprika', 'mango', 'onion', 'lettuce', 'olive', 'bell pepper', 'rutabaga', 'raspberry', 'banana', 'pomegranate', 'parsnip', 'lemon', 'strawberry', 'unknown', 'pear', 'mushroom']

Fruits: ['watermelon', 'cherry', 'apple', 'orange', 'raspberry', 'jackfruit', 'papaya', 'cherry', 'grapes', 'orange', 'kiwi', 'plum', 'strawberry', 'mango', 'raspberry', 'banana', 'pomegranate', 'lemon', 'strawberry', 'pear']

Vegetables: ['beetroot', 'sweetpotato', 'cauliflower', 'cabbage', 'corn', 'chilli pepper', 'bell pepper', 'cucumber', 'carrot', 'ginger', 'peas', 'garlic', 'potato', 'eggplant', 'paprika', 'onion', 'lettuce', 'bell pepper', 'rutabaga', 'parsnip', 'mushroom']

Not a type of produce: ['unknown', 'soy beans', 'jalepeno', 'raddish', 'olive', 'unknown']

The two unknowns are a picture of professor choo and a video game image of sweet corn.

The jalapeño is a picture of the Jeff Dunham puppet José Jalapeno on a stick.

The soybean picture has two hands cupping the soybeans. The hands take two thirds of the picture.

The olive and radish images are either sorted in error or I choose a minority of the images.

Models Utilized

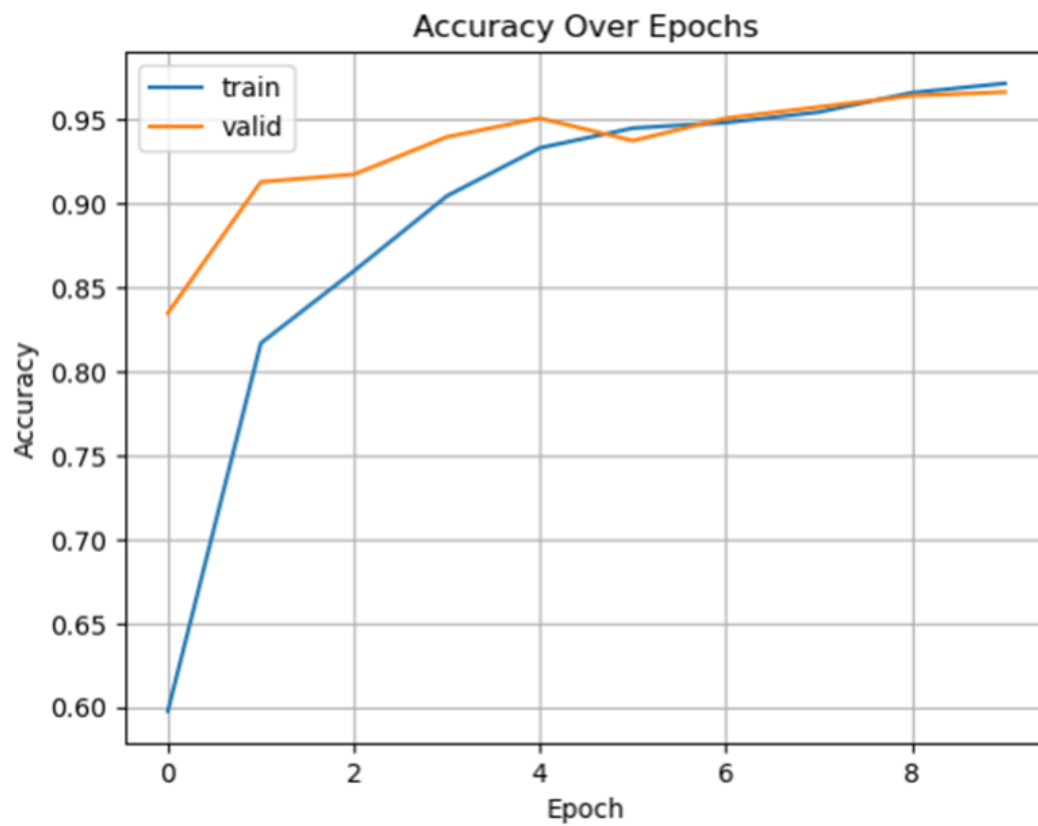
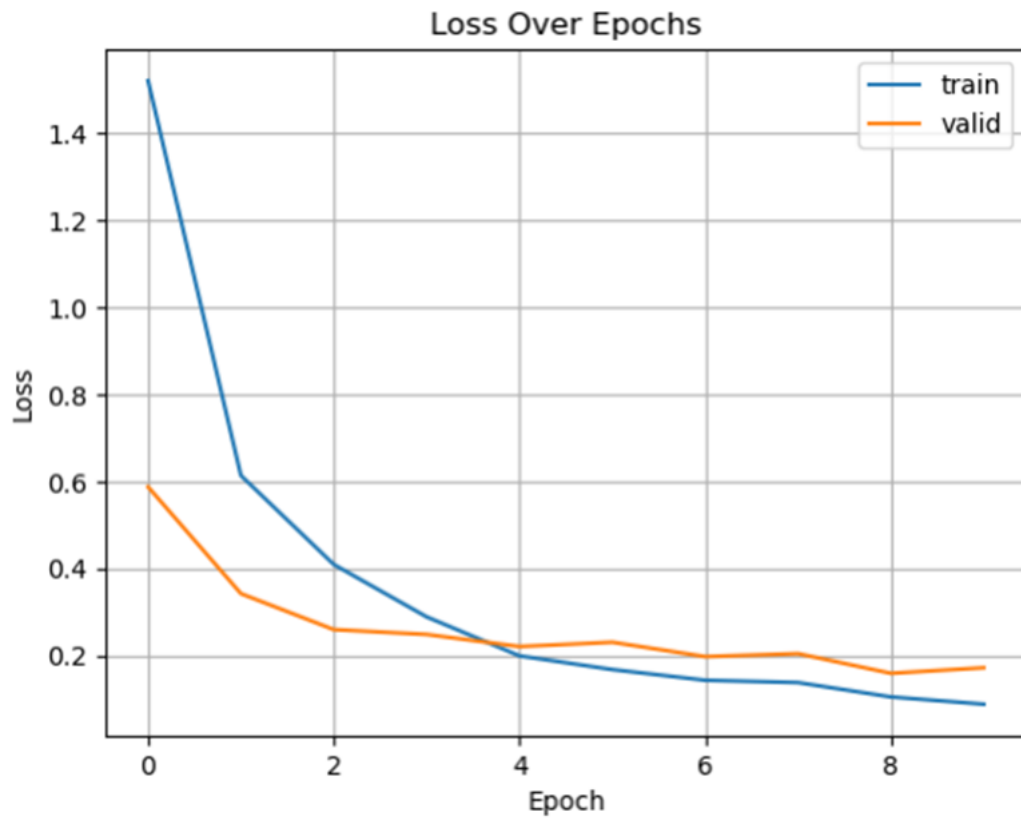
The models I used are Convolutional Neural Network (CNN), Deep Neural Network (DNN), and (MLP) Multilayer Perceptron.

See Baseline Model section for CNN baseline.

To convert this into a DNN baseline. The custom layers by adding a `base_model.output`, `GlobalAveragePooling2D`, and `Dense` set at 1024 with an activation of `Relu`. Compute the predictions. Set the model inputs and outputs. Compile the model with the optimizer set to `Adam`, loss set to `categorical_crossentropy`, and metrics set to `accuracy`. The `Adam` optimizer is an extended version of stochastic gradient descent used for deep learning. Then the augmentation is set into an `if` loop for a complex network.

The results for the DNN baseline:

	Precision	Recall	F1-score	Support
Accuracy			0	474
Macro Average	0	0	0	474
Weighted Average	0	0	0	474



Fruits: ['watermelon', 'cherry', 'apple', 'orange', 'raspberry', 'jackfruit', 'papaya', 'cherry', 'grapes', 'orange', 'kiwi', 'plum', 'strawberry', 'mango', 'blackberry', 'banana', 'pomegranate', 'lemon', 'strawberry', 'pear']

Vegetables: ['beetroot', 'sweetpotato', 'cauliflower', 'cabbage', 'corn', 'chilli pepper', 'bell pepper', 'cucumber', 'carrot', 'ginger', 'peas', 'garlic', 'potato', 'eggplant', 'paprika', 'onion', 'lettuce', 'bell pepper', 'rutabaga', 'parsnip', 'mushroom']

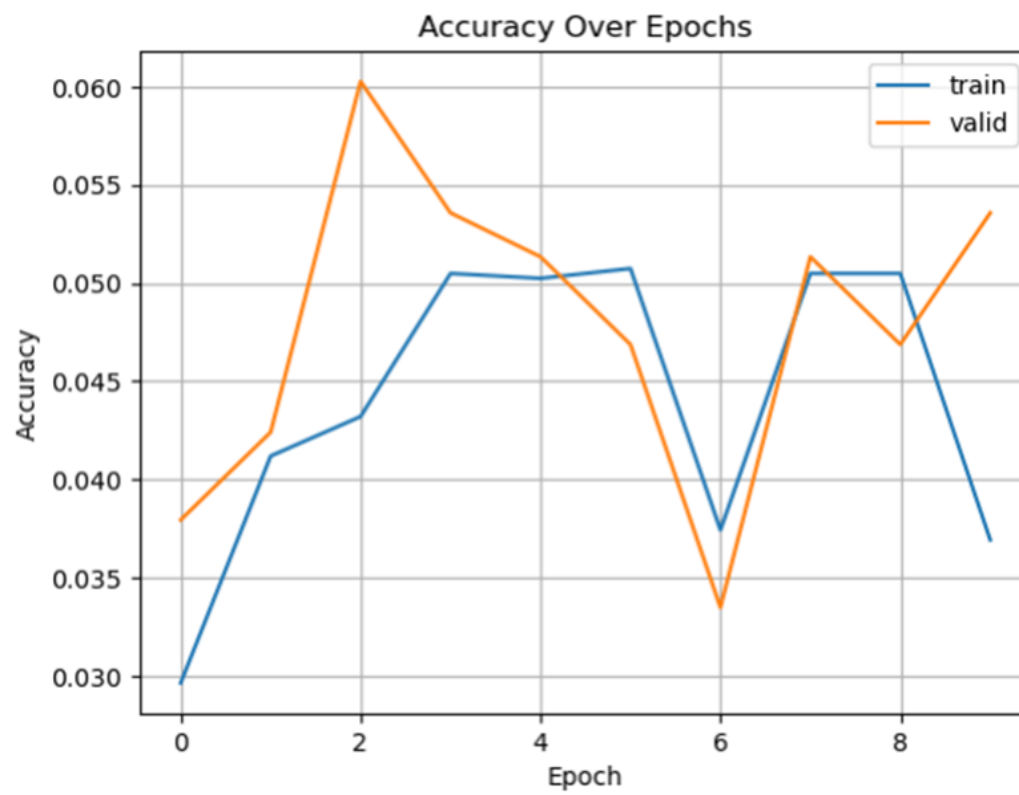
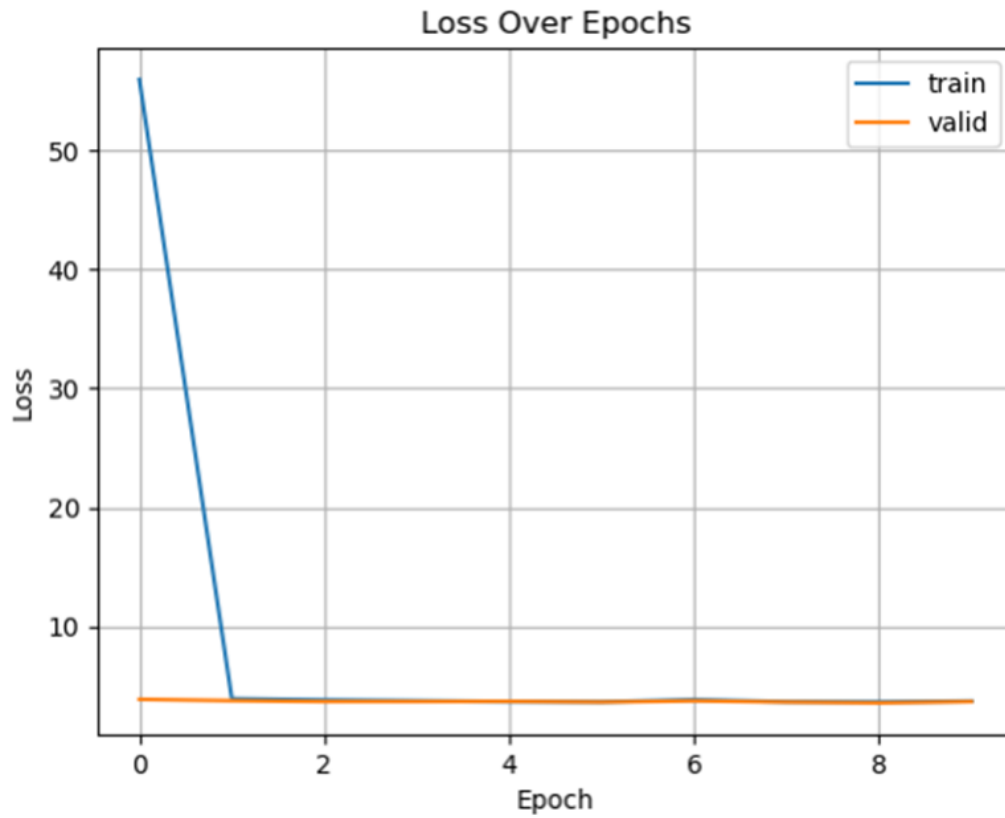
Not a type of produce: ['unknown', 'soy beans', 'jalepeno', 'raddish', 'olive', 'unknown']

The DNN baseline did not prove favorable in accuracy but did sort correctly.

To convert this into a MLP baseline. I just built a MLP model. It's sequential. Flatten the image. Added three dense layers. Two of them set to 1024 and 512 with both set to a ReLu activation. The last layer is set with the num_classes with a softmax activation.

The results for the MLP baseline:

	Precision	Recall	F1-score	Support
Accuracy			0.05	474
Macro Average	0.05	0	0.01	474
Weighted Average	0.86	0.05	0.09	474



['tomato', 'peas', 'peas', 'tomato', 'tomato', 'tomato', 'tomato', 'tomato', 'peas', 'peas', 'onion', 'peas', 'tomato', 'tomato', 'tomato', 'tomato', 'peas', 'peas', 'tomato', 'peas', 'peas', 'peas', 'peas', 'peas', 'peas', 'peas', 'peas', 'peas', 'peas', 'tomato', 'tomato', 'tomato', 'soy beans', 'peas', 'soy beans', 'soy beans', 'peas', 'peas', 'soy beans', 'tomato', 'peas', 'peas', 'tomato', 'peas', 'peas', 'soy beans']

MLP Final Results

What are these images? ['tomato', 'peas', 'peas', 'tomato', 'tomato', 'tomato', 'tomato', 'tomato', 'peas', 'peas', 'onion', 'peas', 'tomato', 'tomato', 'tomato', 'tomato', 'peas', 'peas', 'tomato', 'peas', 'peas', 'peas', 'peas', 'peas', 'peas', 'peas', 'peas', 'peas', 'peas', 'tomato', 'tomato', 'tomato', 'soy beans', 'peas', 'soy beans', 'soy beans', 'peas', 'peas', 'soy beans', 'tomato', 'peas', 'peas', 'tomato', 'peas', 'peas', 'soy beans']

Which of these are vegetables, fruits, or not a type of produce?

Fruits: []

Vegetables: ['tomato', 'peas', 'peas', 'tomato', 'tomato', 'tomato', 'tomato', 'tomato', 'peas', 'peas', 'onion', 'peas', 'tomato', 'tomato', 'tomato', 'tomato', 'peas', 'peas', 'tomato', 'peas', 'peas', 'peas', 'peas', 'peas', 'peas', 'peas', 'peas', 'peas', 'tomato', 'tomato', 'tomato', 'peas', 'peas', 'peas', 'tomato', 'peas', 'peas', 'tomato', 'peas', 'peas']

Not a type of produce: ['soy beans', 'soy beans', 'soy beans', 'soy beans', 'soy beans']

The MLP baseline did not prove favorable and misidentified.

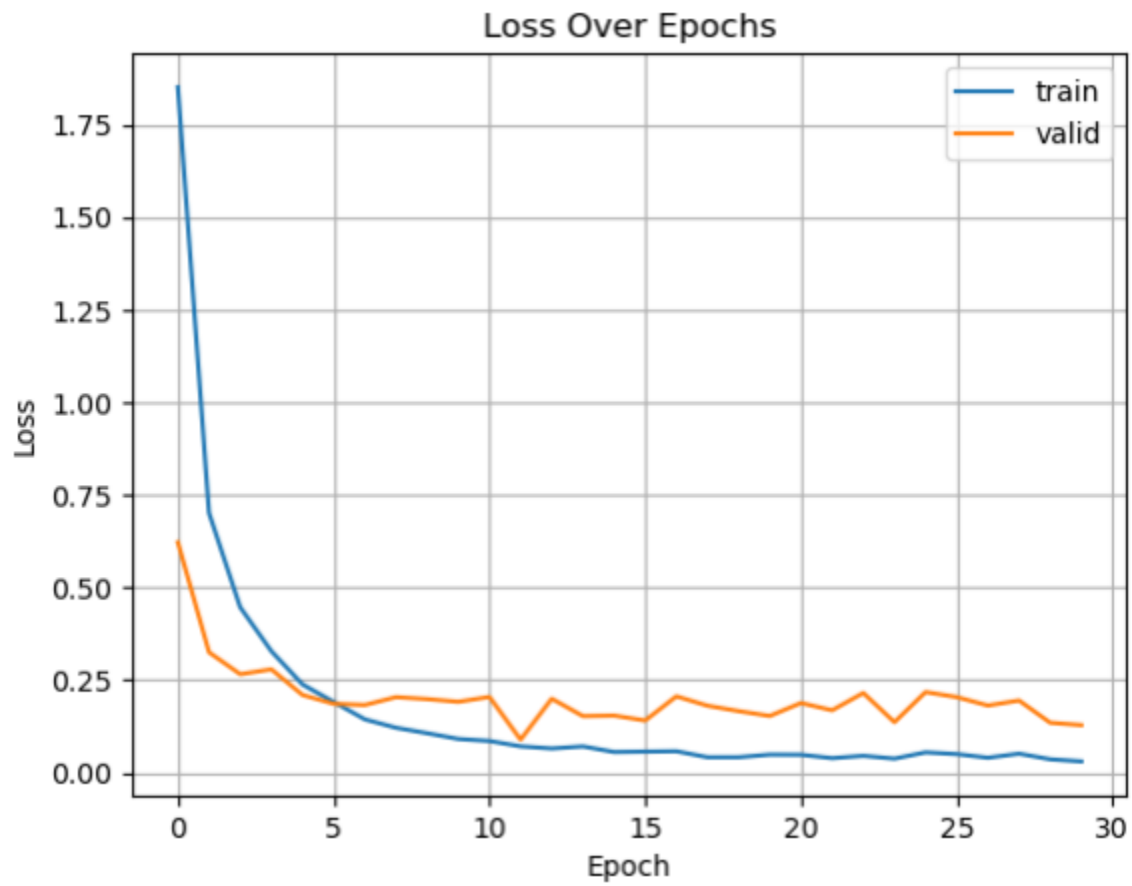
Model Improvements

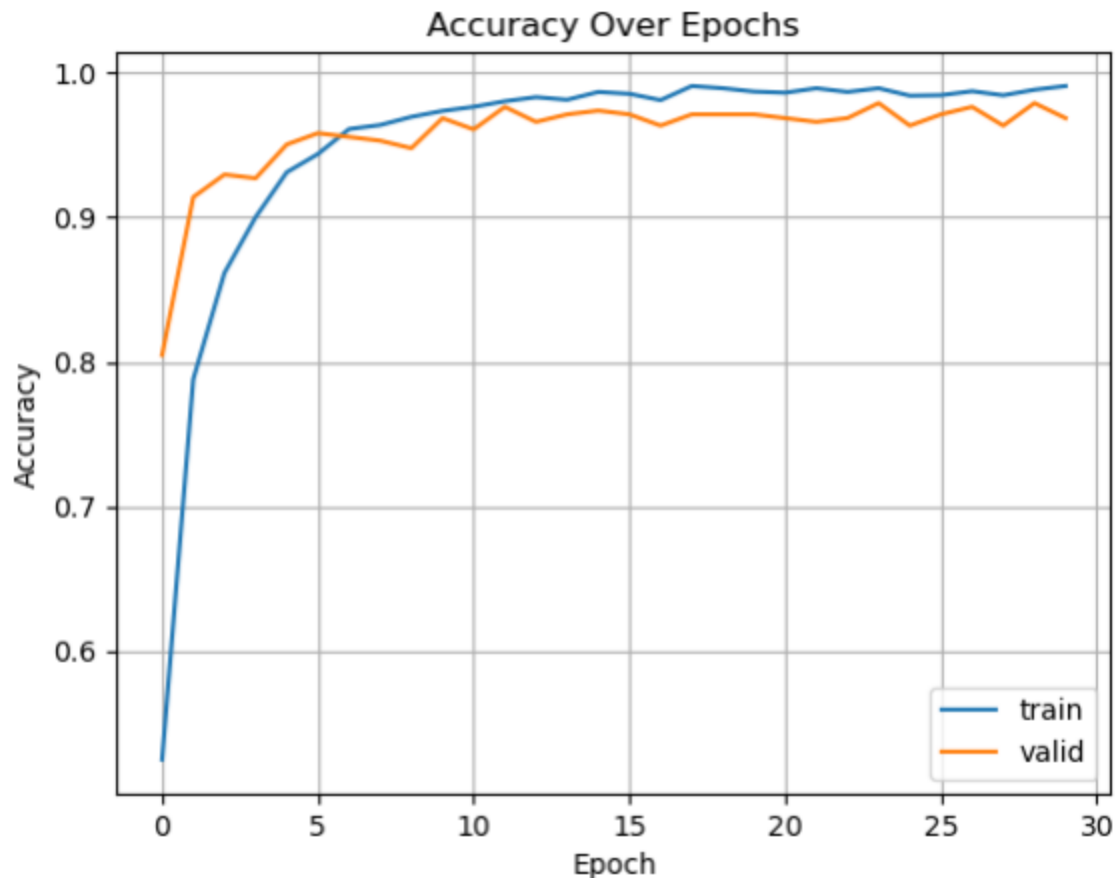
Based on our homework number three, four, and five. I changed the epoch size to 30 and the batch size to 128 with the thought of more to sample and more to test for better results.

I did adjust layers and augment resolution but the results got just as bad as the baseline for the RNN and MLP.

CNN Results

	Precision	Recall	F1-Score	Support
Accuracy			0.97	474
Macro Average	0.97	0.97	0.97	474
Weighted Average	0.98	0.97	0.97	474





CNN Final Results

What are these images? ['watermelon', 'cherry', 'unknown', 'apple', 'orange', 'beetroot', 'raspberry', 'sweetpotato', 'cauliflower', 'cabbage', 'soy beans', 'corn', 'jackfruit', 'chilli pepper', 'papaya', 'bell pepper', 'cucumber', 'carrot', 'cherry', 'ginger', 'jalepeno', 'peas', 'grapes', 'garlic', 'orange', 'kiwi', 'raddish', 'plum', 'potato', 'eggplant', 'strawberry', 'paprika', 'mango', 'onion', 'lettuce', 'olive', 'capsicum', 'rutabaga', 'blackberry', 'banana', 'pomegranate', 'parsnip', 'lemon', 'strawberry', 'blackberry', 'pear', 'mushroom']

Which of these are vegetables, fruits, or not a type of produce?

Fruits: ['watermelon', 'cherry', 'apple', 'orange', 'raspberry', 'jackfruit', 'papaya', 'cherry', 'grapes', 'orange', 'kiwi', 'plum', 'strawberry', 'mango', 'blackberry', 'banana', 'pomegranate', 'lemon', 'strawberry', 'blackberry', 'pear']

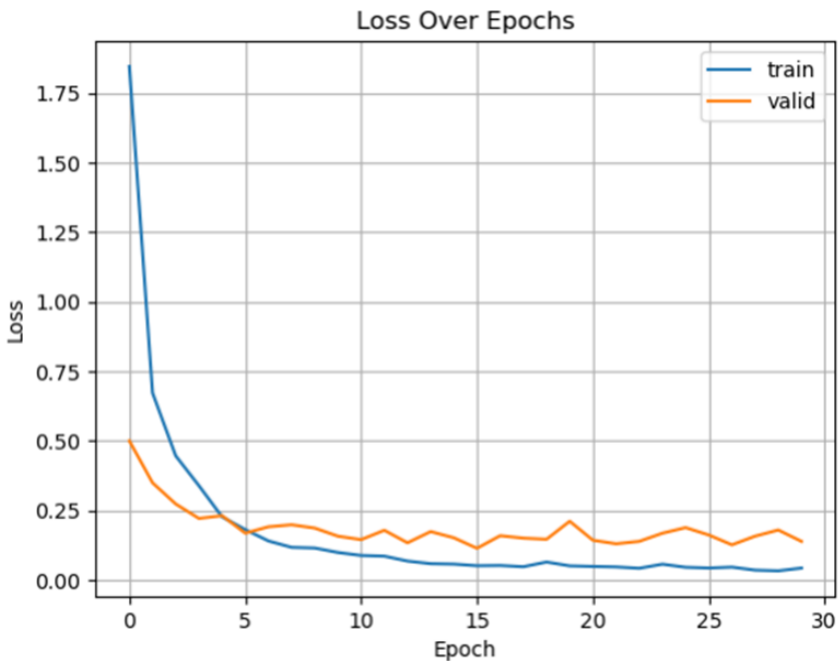
Vegetables: ['beetroot', 'sweetpotato', 'cauliflower', 'cabbage', 'corn', 'chilli pepper', 'bell pepper', 'cucumber', 'carrot', 'ginger', 'peas', 'garlic', 'potato', 'eggplant', 'paprika', 'onion', 'lettuce', 'capsicum', 'rutabaga', 'parsnip', 'mushroom']

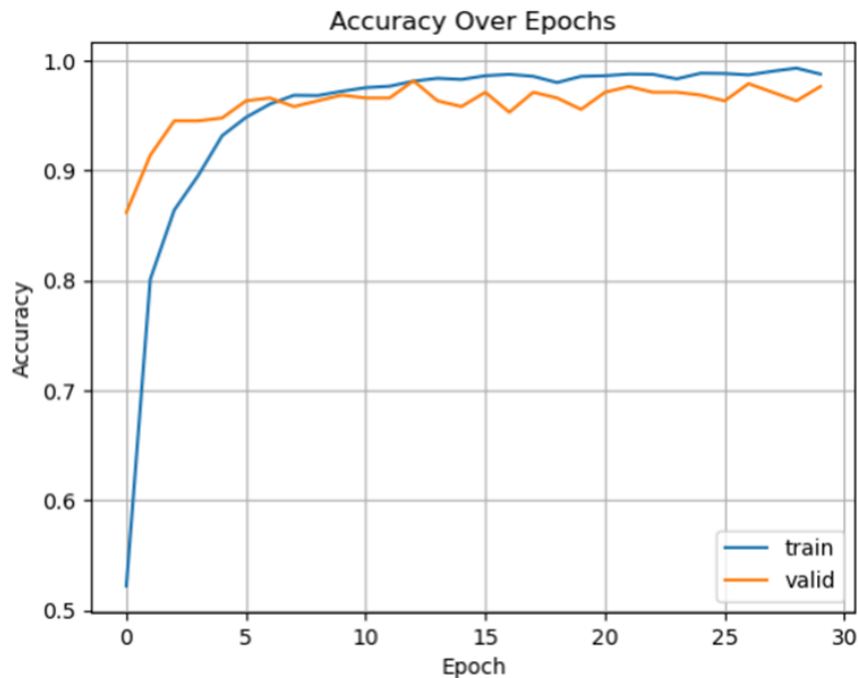
Not a type of produce: ['unknown', 'soy beans', 'jalepeno', 'raddish', 'olive']

DNN Results

	Precision	Recall	F1-Score	Support
Accuracy			0	474
Macro Average	0	0	0	474
Weighted Average	0	0	0	474

DNN Final Results





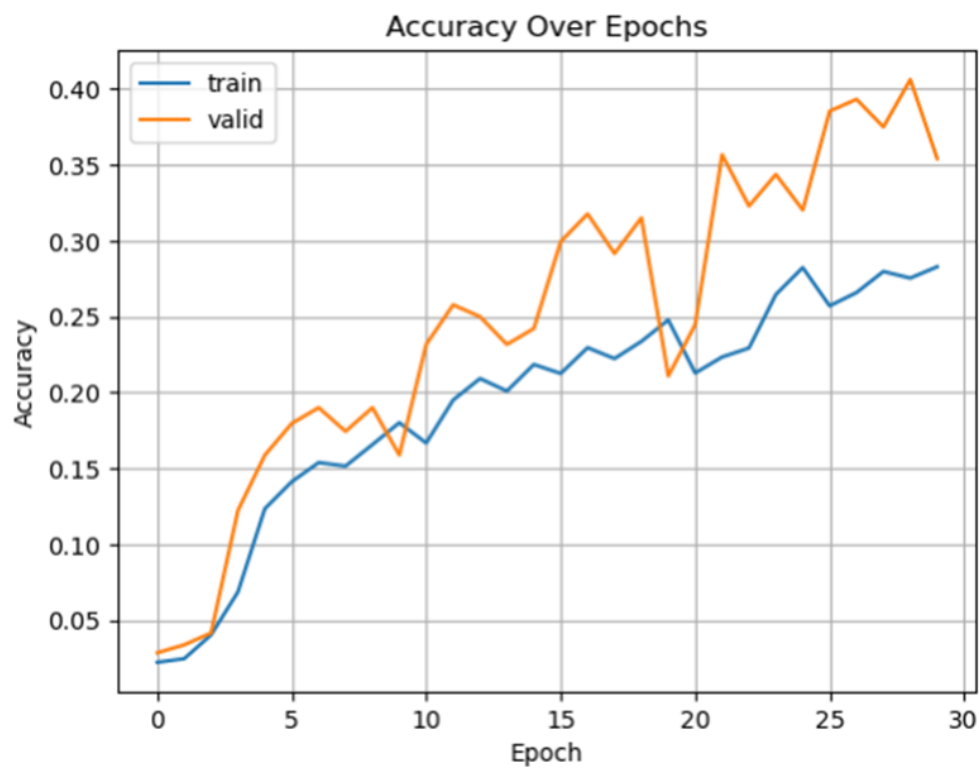
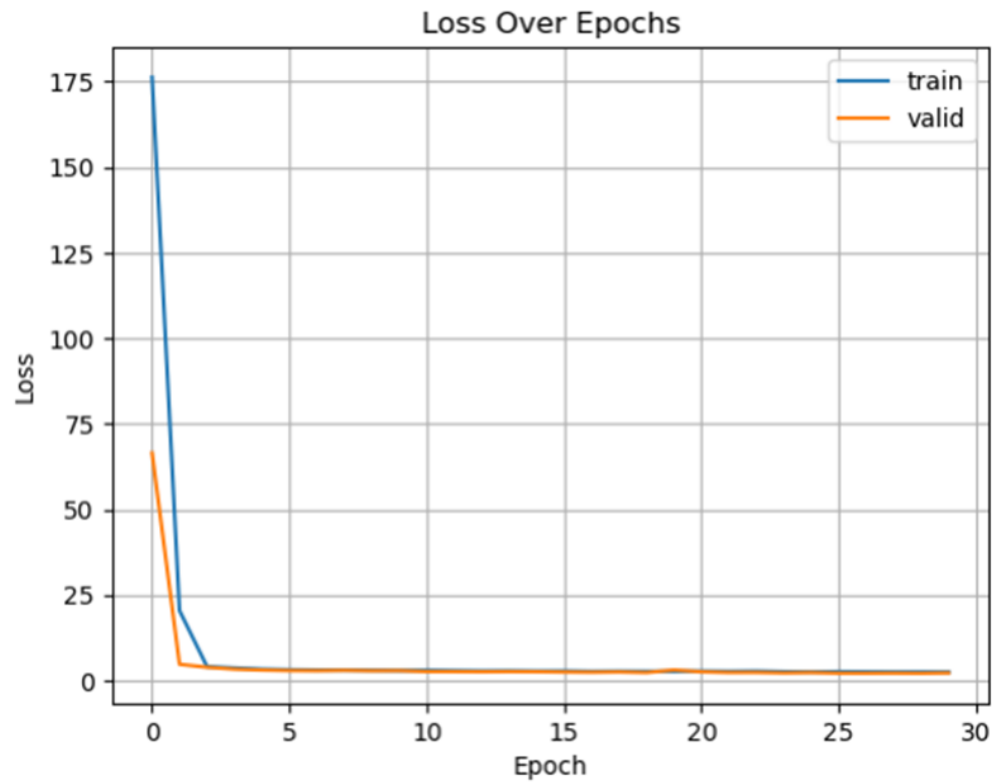
Which of these are vegetables, fruits, or not a type of produce?

Fruits: ['watermelon', 'cherry', 'apple', 'orange', 'raspberry', 'jackfruit', 'papaya', 'cherry', 'grapes', 'orange', 'kiwi', 'plum', 'strawberry', 'mango', 'blackberry', 'banana', 'pomegranate', 'lemon', 'strawberry', 'pear']

Vegetables: ['beetroot', 'sweetpotato', 'cauliflower', 'cabbage', 'corn', 'chilli pepper', 'bell pepper', 'cucumber', 'carrot', 'ginger', 'peas', 'garlic', 'potato', 'eggplant', 'paprika', 'onion', 'lettuce', 'capsicum', 'rutabaga', 'parsnip', 'mushroom']

Not a type of produce: ['unknown', 'soy beans', 'jalepeno', 'raddish', 'olive', 'unknown']

MLP Results



	Precision	Recall	F1-Score	Support
Accuracy			0.35	474
Macro Average	0.34	0.31	0.30	474
Weighted Average	0.51	0.35	0.39	474

['watermelon', 'cucumber', 'garlic', 'apple', 'parsnip', 'pomegranate', 'tomato', 'potato', 'cauliflower', 'cabbage', 'sweetcorn', 'sweetcorn', 'sweetcorn', 'pomegranate', 'soy beans', 'strawberry', 'cucumber', 'onion', 'strawberry', 'soy beans', 'kiwi', 'cabbage', 'grapes', 'garlic', 'kiwi', 'kiwi', 'turnip', 'turnip', 'ginger', 'eggplant', 'strawberry', 'paprika', 'lemon', 'onion', 'pear', 'kiwi', 'soy beans', 'onion', 'raddish', 'pear', 'apple', 'cauliflower', 'turnip', 'sweetpotato', 'soy beans', 'ginger', 'potato']

MLP Final Results

What are these images? ['watermelon', 'cucumber', 'garlic', 'apple', 'parsnip', 'pomegranate', 'tomato', 'potato', 'cauliflower', 'cabbage', 'sweetcorn', 'sweetcorn', 'sweetcorn', 'pomegranate', 'soy beans', 'strawberry', 'cucumber', 'onion', 'strawberry', 'soy beans', 'kiwi', 'cabbage', 'grapes', 'garlic', 'kiwi', 'kiwi', 'turnip', 'turnip', 'ginger', 'eggplant', 'strawberry', 'paprika', 'lemon', 'onion', 'pear', 'kiwi', 'soy beans', 'onion', 'raddish', 'pear', 'apple', 'cauliflower', 'turnip', 'sweetpotato', 'soy beans', 'ginger', 'potato']

Which of these are vegetables, fruits, or not a type of produce?

Fruits: ['watermelon', 'apple', 'pomegranate', 'pomegranate', 'strawberry', 'strawberry', 'kiwi', 'grapes', 'kiwi', 'kiwi', 'strawberry', 'lemon', 'pear', 'kiwi', 'pear', 'apple']

Vegetables: ['cucumber', 'garlic', 'parsnip', 'tomato', 'potato', 'cauliflower', 'cabbage', 'sweetcorn', 'sweetcorn', 'sweetcorn', 'cucumber', 'onion', 'cabbage', 'garlic', 'turnip', 'turnip', 'ginger', 'eggplant', 'paprika', 'onion', 'onion', 'cauliflower', 'turnip', 'sweetpotato', 'ginger', 'potato']

Not a type of produce: ['soy beans', 'soy beans', 'soy beans', 'raddish', 'soy beans']

My overall observation is after I increased the epoch and batch size: CNN gained a two percent gain in accuracy; DNN remained the same; MLP went from zero percent to thirty percent in most cases and fifty percent in accuracy and correctly sorted the images.

In ranking from best to worst neural network after improvements are as follows: CNN, MLP, and DNN.

Discussions & Conclusions

My overall observation is after I increased the epoch and batch size: CNN gained a two percent gain in accuracy; DNN remained the same; MLP went from zero percent to thirty percent in most cases and fifty percent in accuracy and correctly sorted the images.

In ranking from best to worst neural network after improvements are as follows: CNN, MLP, and DNN.

I would not know of any future suggestions to recommend. What I wanted to but did not have time for are: better understand how to program an error catch based on accuracy calculations, output images of each listed to ensure what is listed is what is outputted, have the input be a camera and the code can actively identify what produce it can see.

The issues I have are the code takes a long while to process and learn to debug. DNN took the longest to program. Originally, I was trying to code a pop-out prompt to have the user upload the image to test, but the directory approach turned out more favorable since it can test multiple images at once. If I had more time, accommodating all picture types would have been great.

The main code I used as a one to one is from the “Fruits and Vegetables Image Recognition Dataset.”.

I used Chat GPT to help with debugging the code.

The rest in the work cited area are the resources that helped me form and understand terms and how to code in Python.

Ultimately, I’m glad I was able to set forth and accomplish what I proposed.

References

Kritik Seth, "Fruits and Vegetables Image Recognition Dataset," Kaggle 2020
[<https://www.kaggle.com/kritikseth/fruit-and-vegetable-image-recognition>]

"ChatGPT." ChatGPT, OpenAI, 2 September 2023.

[Ogle] EE258 Class Homework#3 Report., Summer 2023.

[Ogle] EE258 Class Homework#4 Report., Summer 2023.

[Ogle] EE258 Class Homework#5 Report., Summer 2023.

NumPy. "NumPy Documentation." NumPy, numpy.org/doc/stable/index.html. Accessed 2 September 2023.

Python Software Foundation. "Python 3 Documentation." Python.org, docs.python.org/3/. Accessed 2 September 2023.

MLP help

scikit-learn Contributors. "sklearn.neural_network.MLPClassifier." scikit-learn Documentation,
scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html.
Accessed 2 September 2023.

Author Not Specified. "Multi-Layer Perceptron (Neural Network) using Python." Machine Learning Geek,
machinelearninggeek.com/multi-layer-perceptron-neural-network-using-python/.
Accessed 2 September 2023.

DNN help

Brownlee, Jason. "Your First Deep Learning Project in Python with Keras Step-By-Step." Machine Learning Mastery,

machinelearningmastery.com/tutorial-first-neural-network-python-keras/. Accessed 2 September 2023.

OpenCV. "Table of Contents - Deep Neural Networks (dnn) module." OpenCV Documentation, docs.opencv.org/4.x/d2/d58/tutorial_table_of_content_dnn.html. Accessed 2 September 2023.